Catch the Eavesdropper — BB84 Blue Track Challenge

Overview

Teams write Bob's side of BB84 quantum key distribution and must turn a noisy, possibly eavesdropped quantum transmission into a provably secure shared key, or abort. Alice runs on the organisers' server, so every bit Bob learns from her is logged and scored.

What students learn: sifting, statistical parameter estimation with confidence bounds, Cascade error correction, Toeplitz privacy amplification, and the central lesson of QKD: noise and eavesdropping are indistinguishable, so every error is paid for as if Eve caused it.

Format: teams of 3–4, 24 hours of hacking, 10 hidden sessions, automatic scoring plus a 5-minute final presentation. Python is recommended; any language that speaks HTTP works. No quantum hardware or quantum SDK is needed.

Prerequisites: basic probability and Python. A 45-minute opening lecture covers the BB84 protocol and the key-length formula.

Student brief

Implement BB84 with noise and/or an eavesdropper.

The full protocol is:

Initial exchange and shift

1. Alice generates N random bits (the bases) and N radom bits (the qubit states). Then goes through the two arrays in tandem and if the basis is 0 and the qubit is 0 sends |0>, if the basis is 0 and the qubit is 1 sends |1>, if the basis is 1 and the qubit is 0 sends |+>, if the basis is 1 and the qubit is 1 sends |->. The sending is done over the quantum channel. Alice stores both arrays
2. Bob generates N random bits (the bases). For each qubit received checks his base and measures: if the base is 0 measures in Z, if it's 1 measures in X. Stores the result
3. Alice sends the N bases bits she generated over the public channel
4. Bob checks them against his generated bases and discards both the bases and measured results in the positions where the bases don't match. n matching key bits remain.

Estimate the noise

5. Allice generates k random integers (the positions) and sends a) the array of k positions, b) the key bits at the k positions to Bob over the public channel.
6. Bob sends the key bits at the k positions to Alice over the pubic channel
7. They both count the mismatches m and compute Q_hat = m/k. They also discard the key bits at those positions.

8. They can estimate a better Q by choosing a common epsilon (pre-agreed=10^10) and using Hoeffding or Clopper–Pearson methods:
   Hoeffding:
   $$Q_u = \hat{Q} + \sqrt{\frac{\ln(1/\epsilon)}{2k}}.$$
   Clopper–Pearson:
   `scipy.stats.beta.ppf(1-eps, m+1, k-m)`.
   If Q_u exceeds a threshold (≈11% for this protocol), abort **here**

Error correction (Cascade)

9. They split the remaining key into blocks of length $b \approx 0.73/\hat{Q}$.
   For each block
   1. Alice computes the parity and sends it to Bob over the public channel
   2. Bob computes the parity and if it agrees confirms
   3. if the confirmation arrives Alice moves on to the next block, otherwise splits the block in half and goes to step 9.1. When a block of length 1 has a mismatch Bob flips the bit and reports this to Allice
10. Alice chooses a random permutation , permutes the key bits accordingly and sends the permutation to Bob who also permutes the key bits
11. Repeat steps 9,10 three times and record how many parity bits have been exchanged. Call the number leak$_{EC}$
12. Alice computes a small hash of the key and sends the hast to Bob over the public channel. Bob computes the same hash (pre-agreed) and if correct confirms to Alice if not signals missmatch.
13. Alice: If mismatch is signaled go to step 9 else move on. (If third missmatch discard the key and signal this to Bob)
14. Alice computes
    $$\ell = n - n\,h(Q_u) - \text{leak}_{EC} - 2\log_2 \frac{1}{\epsilon_{PA}}.$$
    where Q_u is computed at step 8. Leak_EC is computed at step 9, epsilon_PA=10^10. And h(x) is given by
    $$h(x) = -x\log_2 x - (1-x)\log_2(1-x).$$
    If l < 0 assume someone is listening, discard the key and start over. If not she generates n

random bits and l random bits and sends them to Bob over the public channel. From these they can both compute the T matrix and the final key by multiplying the T matrix and the key.

15. Use the key to encrypt the message and send the encrypted message over the public chanel

Someone needs to build a server that acts as the communication interface: a quantum channel and a classical public channel. It can be built by the organizers or left to the participants. The server can inject noise by randomly flipping qubits at a given error rate.

Parameters that can be changed:

1. Initial random exchange size N. Typical value 200000
2. The number of positions in the noise estimation k. Typical values 10000-30000.
3. Qubit error rate. Typical values 1%-4%
4. Eve can measure some qubits and re-send them. This will add some more noise. The percent of qubits measured by Eve can be set. Typical value 0%-1%